Availability & Time conflict detection

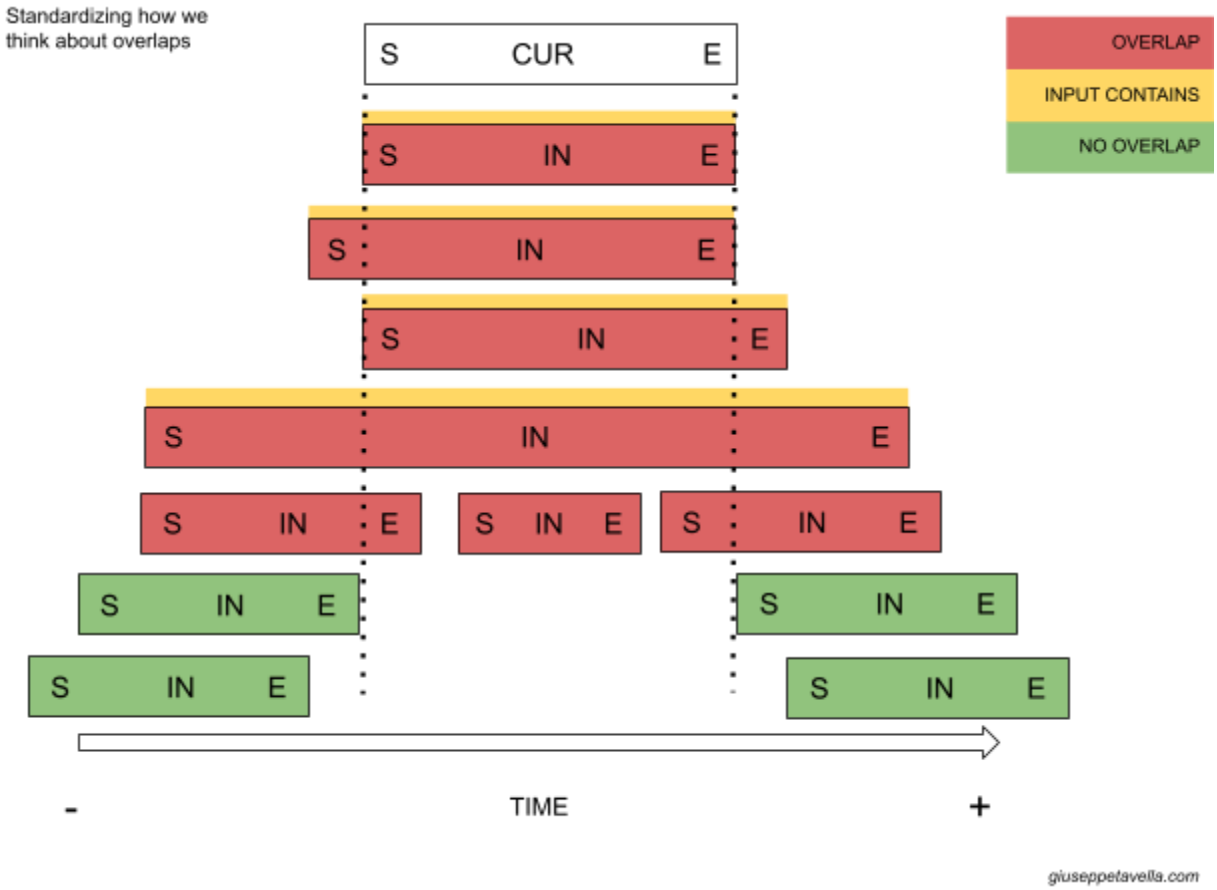
Algorithm

Github repo with diagram

This is my attempt to standardize and generalize algorithms that deal with availability, overlap and conflict detection. The problem came up for me when designing complex SQL queries, while building agenda and appointment systems that were supposed to replace the entire company's appointment system (a mix of Google Calendar, Excel and manual tracking), and that the company would have to rely on, as the internal and only source of truth. And it did. My experience with such systems is that it's a serious business where getting a sign wrong means your algorithm is unreliable and hard to debug. There's no "it works 80% of the time, it's good enough". Either it's logically correct, or it's not. The unforgiveness aspect of this problem calls for standardization and more formal reasoning. Call this a specification, a problem-solving template or generic algorithm - it includes:

- Availability and conflict detection algorithm that you can implement in your workflow, in a way that is language-agnostic and implementation-independant.
- We start from one simple formula and derive other formulas with formal reasoning, not intuition.
- Why focusing on availability is more intuitive than wrapping your head around conflicts.
- Visual representation of all possible overlaps.
- Simple, concise syntax to describe ranges, so we don't have to worry about naming things.
- Why dates and times behave differently in ranges.
- Terminology and synonims.

This is the visual representation of all overlaps and no overlaps.



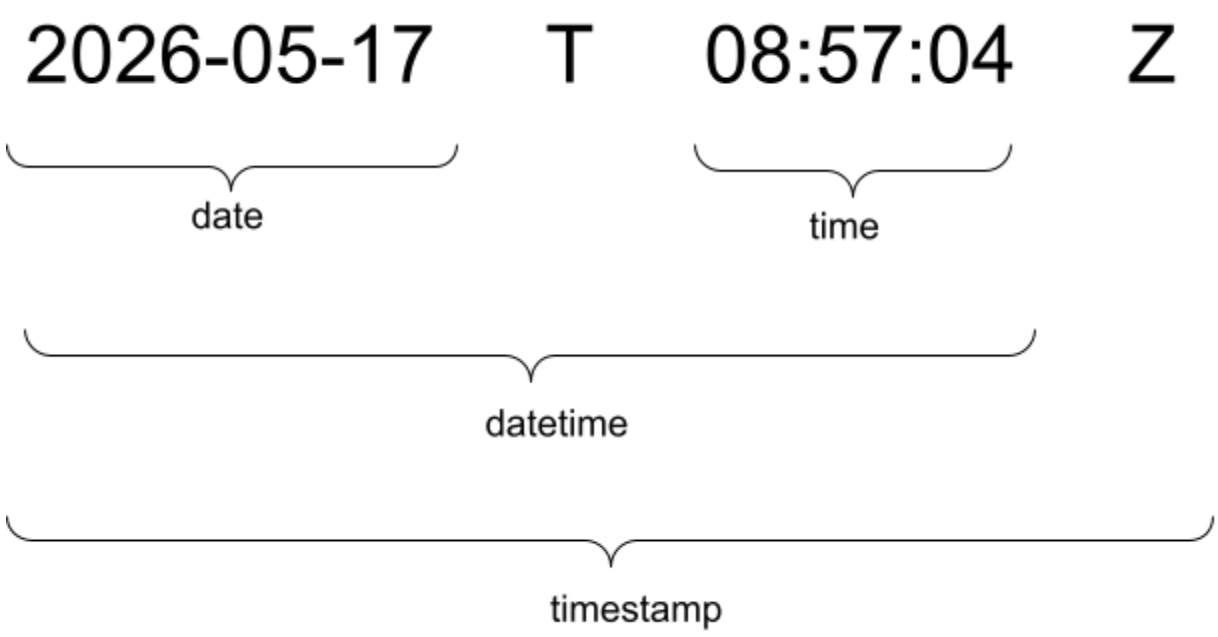
Date or time

Let's address the first terminology issue. When we talk about time in day to day life, what do we mean exactly? A day? A year? The problem is that we use the word time loosely, to mean:

- A specific point in time.
- To indicate an hour of day, maybe with minutes, for example hour 18:23.
- Do we mean that a certain event recurs at a given time, say every day, or that it happened at a time?

Here's the thing: As a property of an object, time refers to an hour. As a general topic of conversation, time refers to "everything that has to do with time, so dates, dates plus time, timestamps with and without time zones etc.".

It also helps to remind ourselves of the ISO 8601 standard which, in short, breaks down time as shown in the image below.



From this, we gather that:

- *Date has context*. You can say that something happened on that date. It makes sense, a date is a specific point in time, though not very specific, but you know what day it was or will be. Same applies for a datetime, which is more specific.

- *Time has no context*. And because it has no context, it can be used to also mean “recurring” at a given time. Saying that something occurred at 14:24 hour doesn’t mean anything. The question will always be, on what date? Because a date has context, a time does not. However, you can say that an event recurs at 14:00 hour every day, but you can’t say that an event recurs at the 2025-06-01 date.

- When we talk about time or datetime, we can *leave out the minutes*, assuming it’s a level of detail we’re not interested in. But internally a system will simply set them to 0. Thus, 13:54 is actually 13:54:00 hour.

- By day we could mean date, or also a weekday (monday, tuesday etc.). Because in this specification this distinction is not relevant, we could use day and date interchangeably. However, if we're also dealing with days of the week, we might do well to define date as a date, and *day as day of the week*.

Dates excluded, times included

Let's also observe the following, which will impact our resulting algorithm:

- ***When testing for availability with dates, the input range must exclude existing dates***. My work contract ends on 2026-05-31. Can I have a new contract starting on the same date? No, because when we talk about the end of a date range, we mean the end date is like any other date in that contract that is currently busy. Therefore, when I'm testing for availability with dates, the input range must start the very next day, which is why the input range's start must be strictly greater than the existing range's end. Same applies for the start. If my contract started at 2025-05-01, it means that before that date I could have had another contract. But whatever that contract was, it had to end before 2025-05-01, which is why we use the strictly less than comparison (See Github repo for visual representation of overlaps). Thus, when testing for availability with dates, if I were to allow the input start to be equal to the existing end, or the input end equal to the existing start, I would effectively be creating a conflict, which is why we use strict comparison, i.e. without equal. **Logical error if we include dates when testing for availability**: We've created a conflict, the new date range overlaps with an existing date range. We're saying that a candidate date range is available or free, when it isn't.

- ***When testing for availability with times, the input range must include existing times***. The doctor can have an existing appointment ending at 13:00 hour and a new appointment starting at 13:00 hour. Similarly, the doctor can have an existing appointment starting at 15:00 hour and I can book from whatever time before that but ending exactly at 15:00 hour. Makes sense, right? It sounds obvious, but our algorithm will have to take this into account. Let's pretend we didn't include time when testing for

availability. I'm on the doctor's website and want to see if they're available from 11:00 to 12:00 hour. The doctor currently has an appointment ending at 11:00 hour (and no appointments after that for the entire day), so my appointment is allowed to start at 11:00 hour, and in theory I should be able to book whatever time range starting from exactly 11:00 hour with whatever end. Of course, by whatever end I mean, the allowed time slot, which is 1 hour. In short: I should be able to book from 11:00 to 12:00 hour. However the doctor's appointment system has the following expression: "A new appointment is available if the input range's start is strictly greater than the existing range's end", which is replaced with "*if 11.00 (input start) > 11.00 (existing end), then input range is available, you can book*". Expression evaluates to false: You cannot book. However, you can book starting from the next time slot, because "*if 12.00 (input start) > 11.00 (existing end)*" evaluates to true. If the time slots that I'm allowed to select are 1 hour, that means the doctor will not be booked in that entire 1 hour, because of this logical error. It follows that the bigger the time slot, probably the bigger the negative impact for the end user or company. **Logical error if we exclude times when testing for availability**: We're saying something is unavailable or busy, when it is available.

Synonims

Availability or conflict is an implementation-dependent terminology. We say that the time slot for an appointment is available or unavailable, free or busy. We say the contract can be extended if there's no conflict with the new end date, and so forth. In this specification, I could even use availability or conflict as opposites of each other, but more formally we should be using the terms overlap or no overlap, which are more generic. For practical reasons, I could use some terms interchangeably. The following table helps to clarify synonyms.

SYNONIMS TABLE		
OVERLAP	**< —---------- is a synonym of —---------- >**	**NO OVERLAP**

Unavailable		Available
Busy		Free
Conflict		No conflict

Describing ranges

Remember that we're dealing with ranges, also known as intervals, not specific points in time. We have two ranges:

- The *input range*, also called the new or candidate range, that we're testing for availability or conflict against so called current ranges. The input range can be the parameter of a function or the named parameters passed to a SQL query, for example. The input range can be seen as a new candidate range to be inserted or updated into a system. Therefore, it is not in the system yet, it's precisely what we're testing to see if it can go in the system, so to speak.

- The *current range*, also called existing or saved range, is data that is saved in a system like a database, or even in a variable.

An issue that comes up with this topic, is the terminology to use when talking about ranges. We want a concise, easy to read, language-agnostic syntax, so that we can focus on the core problem and not worry about naming things. So I've come up with a simple syntax:

- *IN*. This is the input range. Also called the new range, candidate range, or input interval. It's what we're testing for whether or not it can go in the system. Thus, it's almost like an external object, from the system's point of view.

- *CUR*. This is the current range. Also called the existing or saved range. It's already in the system. It's what the input range tests *against*.

- *S*. This is the start of the range, regardless of the range.

- *E*. This is the end of the range, regardless of the range.

We treat a range like an object with attributes, just like in OOP. Continuing our syntax definition:

- *IN.S*. The input range's start.

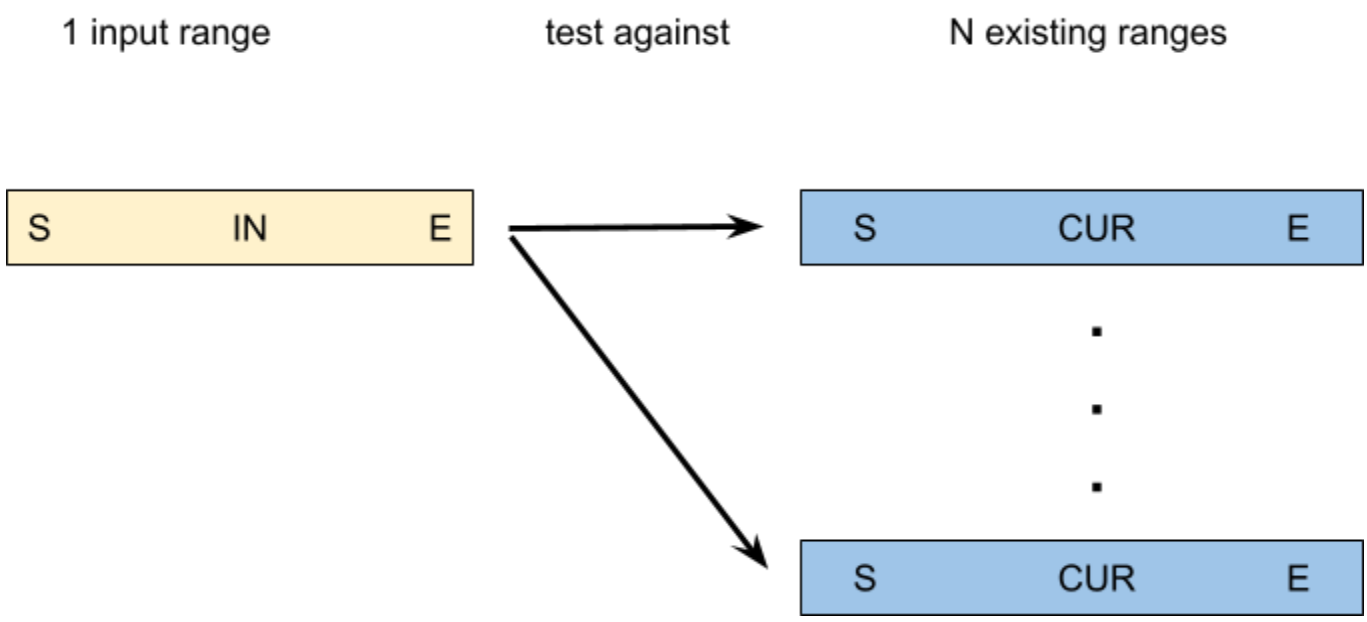
- *IN.E*. The input range's end.

- *CUR.S*. The current range's start.

- *CUR.E*. The current range's end.

1 input range : N existing ranges

At any given moment, we are testing exactly 1 input range against many existing ranges. While it is possible to have 1 input range tested against 1 existing range, that is a subcase of N existing ranges and does not imply structural changes to the algorithm, so we can safely ignore it. In fact, at any given moment, there is exactly 1 input range and N existing ranges, where N >= 0 (the system can be empty for the type of data we're testing, for example there can be no appointments yet). This is why we say "*the* input range" but we say "*an* existing range".



Let's put it into practice with some examples.

```
IN.E < CUR.S
```

"The input range ends before an existing one starts"

Well, that's exactly what it means for a new range to be available, if the expression evaluates to true with a date range for *at least* one existing range!

```
IN.S > CUR.E
```

"The input range starts after an existing one ends"

Well, that's another way to say that the new range is available, if the expression evaluates to true with a date range for *at least* one existing range! We can say to our user "hey, this time slot is available". If you think about it, what have we just said? That if either of these expressions

evaluates to true at least once against an existing range, then the input range is available. Well, that sounds like a logical OR.

```
IN.E < CUR.S
OR IN.S > CUR.E
```

And here we have constructed our first language-agnostic, implementation-independant formula for testing availability with a date range. The explanations above also lead us to a similar formula for testing availability with a time range.

```
IN.E <= CUR.S
OR IN.S >= CUR.E
```

Choosing to focus on availability was a strategic move; It's more intuitive to reason about availability than overlap.

2 states: Either Overlap or No-overlap

If we can agree that there are only two states, then one must be the opposite of the other. Which means, logically negating a state gives us the other state. If there exists an overlap, there cannot exist a no-overlap. If there exists a no-overlap, there cannot exist an overlap. For the same input range and testing against the same existing data, an input range cannot be at the same time available and busy. An input range that represents the time slot for a potential new appointment, cannot be at the same time available and busy, and it also must have a state. If this is true, then we have the following:

- An input range must have a state. An input range cannot somehow "not have a state", it cannot say "I don't have a state, I'm null, I cannot determine the state". It must have a state, which means that the state is *not null* and is *always determinable*. In short, the input range has exactly one non-null state.

- The possible states can be either Overlap or No-overlap.

If we've just stated that the state of an input range exists (is not null and is always determinable), it's exactly one in cardinality and it can either be Overlap or No-overlap, then we're also saying that if it's not one, it's the other. And that means, a logical NOT of a state gives us back the other state, and viceversa.

If, with the formula above, we've come to define availability, and as we said in the Synonims table, availability is a synonym for no-overlap, that means that if we negate no-overlap, we get back overlap, also called conflict. So here's our formula for testing for conflict, overlap or busyness (however you want to call that):

```
NOT (AVAILABLE)
```

It's as simple as that. Now we simply expand the formula for availability with date ranges:

```
NOT (IN.E < CUR.S
     OR IN.S > CUR.E)
```

By applying De Morgan law, we get:

```
NOT (IN.E < CUR.S)
AND NOT (IN.S > CUR.E)
```

We negate the comparison operators, and we get their logical opposites:

```
IN.E >= CUR.S
AND IN.S <= CUR.E
```

And we've just derived the formula for testing for conflict with date ranges that you can integrate in your workflow!

Similary, we can derive the formula for testing for conflict with time ranges.

Conclusions

This was my attempt to standardize some parts of the problem-solving related to this topic, by reducing the cognitive effort due to naming things and naming inconsistencies, facilitating problem-solving through common language, bringing to light logical errors that would otherwise go unnoticed and be hard to debug, and providing a template to solve availability and time conflict-related problems. I hope it was helpful, and any feedback is appreciated. This was version 1.0.0.

May 17th, 2026

Giuseppe Tavella - giuseppetavella.com